

Computer Use at the Edge of the Statistical Precipice

Pierluca D’Oro¹, Sneha Silwal¹, William Wong¹, Yuxuan Sun¹, Fanyi Xiao¹, Manchen Wang¹, Eric Gan¹, Allen Bolourchi¹, Joseph Tighe¹

¹Meta Superintelligence Labs

Evaluating Computer Use Agents (CUAs) on interactive environments is fraught with methodological pitfalls that the field has yet to systematically address. We show that a 1MB replay script that blindly executes a recorded action sequence without ever observing the screen outperforms frontier models on prominent static benchmarks, and prove that its expected success rate is exactly equal to the source agent’s pass@ k in deterministic environments. We trace this and other failures to two root causes: non-principled environment design (static, unsandboxed, or unreliably verified environments) and non-principled evaluation methodology (naïve aggregation and misuse of pass@ k for stateful UI interactions). To address the first, we propose PRISM, five design principles for CUA environments (privileged verification, realistic environments, integrity-checked configurations, sandboxed execution, and multifactorial variability) and instantiate them in DIGIWORLD, a benchmark of 15 realistic sandboxed mobile applications able to evaluate agents in over 3.2 million verified unique configurations. To address the second, we develop an aggregation framework pairing Wilson score intervals with hierarchical bootstrap, producing confidence intervals that correctly account for the nested structure of CUA benchmarks, as we empirically demonstrate. All together, we show that principled environment design and rigorous evaluation methodology are not optional refinements but prerequisites for meaningful CUA research.

Date: May 2026

Correspondence: pierluca@meta.com, tighej@meta.com



1 Introduction

What does it take for a Computer Use Agent (CUA) benchmark to produce reliable measurements? At minimum, the environment must be realistic, sandboxed, and varied enough to prevent memorization; its configurations must be verified as solvable; and task completion must be checked against internal application state rather than brittle proxies. Upon careful inspection, among existing major CUA benchmarks (Xie et al., 2024; Rawles et al., 2024; Drouin et al., 2024; Koh et al., 2024; Garg et al., 2025; He et al., 2024; Sun et al., 2025; Kong et al., 2025; Ullrich et al., 2025; Liu et al., 2018), *none* of them satisfies all of these design principles.

The consequences of these design gaps are severe and interconnected. Environments that depend on live services, drift unpredictably, making results irreproducible. Toy or synthetic interfaces lack the ecological validity to predict deployment performance. Static environments, where each task begins from a single fixed state, allow agents to succeed via memorized action sequences rather than genuine visual reasoning. As configuration spaces grow through parameterization, the absence of automated integrity checking means that a substantial fraction of tasks may be impossible, incoherent, or trivially pre-solved, silently corrupting aggregate metrics. And reliance on LLM-as-judge or screenshot comparison for verification introduces biases and adversarial vulnerabilities (Zheng et al., 2023). Concurrent work confirms that these failures are not theoretical: systematic audits have shown that major benchmarks can be exploited to achieve near-perfect scores without solving any tasks (Wang et al., 2026), and that such exploitation is already widespread (Stein et al., 2026).

Compounding the environmental problem, *evaluation methodology* introduces its own failures. Reporting a single success rate, often without confidence intervals or with intervals computed as if each trial were an

independent coin flip, produces rankings that are unreliable under replication. The problem is worsened by the widespread misuse of $\text{pass}@k$ (Chen et al., 2021), a metric designed for *stateless* code generation that is fundamentally inappropriate for *stateful* UI interactions.

This paper addresses both the environmental and methodological problems:

1. **Diagnosing pitfalls of current benchmarks.** We empirically demonstrate that, on popular deterministic benchmarks, outrageously small agents (of about 1MB of size) that replay successful action sequences from a frontier model can outperform the corresponding source model. We formalize this observation by establishing that the expected success rate of such a replay agent is exactly equal to the source agent’s $\text{pass}@k$ (Remark 1), revealing that $\text{pass}@k$ on static benchmarks measures memorization capacity rather than agent capability.
2. **PRISM design principles and DigiWorld.** We propose five design principles for CUA environments (*privileged* verification, *realistic* environments, *integrity-checked* configurations, *sandboxed* execution, and *multifactorial* variability; PRISM) that benchmarks must satisfy to produce meaningful evaluations, and we instantiate them in DIGIWORLD, a combinatorial benchmark featuring 15 sandboxed mobile applications across domains. With 387 scenarios and over 4,300 task instances, DIGIWORLD independently varies data profiles (~ 10 per app), visual themes (~ 10), and initial UI states (~ 10), generating over 3.2 million verified unique configurations, which leads to a benchmark robust to replay and memorization.
3. **Hierarchical aggregation framework.** Environment variability is necessary for robust evaluation but, paired with stochasticity from an agent’s actions, requires a rigorous statistical framework for confidence interval computation. Since existing naïve aggregation strategies fail to handle this complexity, we develop a rigorous evaluation methodology that uses Wilson score intervals at the rollout level and a hierarchical bootstrap at the suite level, producing confidence intervals that correctly account for the nested structure of interactive benchmarks. We validate this framework through simulations and use it to evaluate frontier models on DIGIWORLD, showing that it offers better statistical coverage than widespread aggregation techniques and that it allows to rigorously track performance variability along each environmental axis.

Our title is an homage to Agarwal et al. (2021), which exposed analogous statistical failures in deep RL evaluation. Where that work showed that point estimates of mean scores across a handful of Atari games produced unreliable agent rankings, we show that the CUA evaluation ecosystem suffers from a *compounded* version of the same issue, non-principled environments feeding non-principled evaluation methodology, and we propose concrete solutions to both.

2 Common pitfalls of CUA environment and metric design

Existing CUA benchmarks exhibit several recurring environmental weaknesses: unreliable verification via LLM-as-judge (Zheng et al., 2023; Wang et al., 2026), synthetic interfaces that lack ecological validity (Ullrich et al., 2025; Liu et al., 2018), absence of automated integrity checking for generated configurations, dependence on live environments that drift over time (He et al., 2024; Drouin et al., 2024), and static single-configuration setups vulnerable to memorization (Zhang et al., 2018; Cobbe et al., 2020). Compounding these, evaluation *methodology* introduces its own failures: naïve statistical aggregation (Agarwal et al., 2021) and misuse of $\text{pass}@k$ produce rankings that cannot be replicated (Section 2.2). We now demonstrate the two most consequential failures empirically: the vulnerability of static environments to memorization, and the statistical fragility of standard reporting practices.

2.1 Outrageously small agents can succeed on static benchmarks

Setup. When a deterministic benchmark uses a single fixed initial state per task, blindly replaying a previously successful action sequence should succeed at a high rate. We construct a *replay agent*: for each task, we record the action sequence from a successful trajectory produced by a frontier model, then replay that exact sequence on subsequent evaluations without observing the state of the screen. The replay agent performs no visual reasoning; it simply executes the same clicks, taps, and keystrokes in the same order. We evaluate the

Table 1 Success rate of source frontier model and replay agent. Blindly replaying a recorded trajectory achieves high success rates on static benchmarks but collapses on DIGIWORLD.

	OSWORLD	MOBILEWORLD	DIGIWORLD
Frontier model	70.6%	32.7%	45.7%
$\sim$ 1MB-script replay agent	71.1%	41.5%	6.90%

replay agent on two prominent CUA benchmarks, OSWORLD and MOBILEWORLD, and on DIGIWORLD (our own benchmark described in Section 5).

Results. Table 1 shows that the replay agent achieves higher success rate than the corresponding frontier model on static benchmarks, despite never observing the screen. Because the environment resets to the same initial state every time, the same action sequence produces the same outcome (apart from small mismatches due to benchmark flakiness). On DIGIWORLD, where each evaluation presents a novel combination of data profile, theme, and UI state, the replay agent collapses to near zero: the recorded actions no longer correspond to the correct interface elements. This confirms that static benchmarks are largely measuring whether the correct action sequence is *known*, not whether the agent can *reason* about what it sees. See Appendix B for evaluation details.

Formal grounding. The replay agent’s success is not accidental but structurally guaranteed. Consider a deterministic benchmark with N tasks and a stochastic source agent π with success probability p_i on task i . Let π_R be the replay policy constructed from k independent rollouts of π : for each task, π_R stores one successful trajectory (if any) and replays it verbatim. We write $\text{SR}(\cdot)$ for the success rate averaged over tasks.

Remark 1 (Replay Equivalence). *On a deterministic benchmark with fixed initial states, a replay policy that stores and replays one successful trajectory per task (if any exists among k rollouts) achieves an expected success rate equal to the source agent’s $\text{pass}@k$:*

$$\mathbb{E}[\text{SR}(\pi_R)] = \text{pass}@k(\pi).$$

Furthermore, $\text{SR}(\pi) \leq \text{pass}@k(\pi)$ for all $k \geq 1$, with strict inequality whenever $k > 1$ and $p_i \in (0, 1)$ for some task i .

Corollary 2 (Brute-Force Solvability of Static Benchmarks). *For any agent with $p_i > 0$ on all N tasks, $\lim_{k \rightarrow \infty} \mathbb{E}[\text{SR}(\pi_R)] = 1$. Every static benchmark can be solved to arbitrary precision by running a stochastic agent for sufficiently many rollouts and memorizing successful trajectories.*

Proofs are in Appendix A. Remark 1 reveals that the replay agent is the physical embodiment of $\text{pass}@k$: a constructive, memoryless algorithm that converts any nonzero success probability into near-perfect benchmark performance. This establishes a formal link between the environmental vulnerability demonstrated above and the methodological problems discussed next: *reporting $\text{pass}@k$ on a static benchmark is equivalent to reporting the success rate of a blind replay policy.*

Implications. Although our demonstration is purposely extreme, it points to the fact that any benchmark score obtained on a static environment is confounded with the degree to which the agent (or its training data) may have been inadvertently exposed to the specific evaluation trajectories. Improvements on such benchmarks may reflect better memorization rather than better visual grounding or capabilities. This does not mean that these benchmarks are without value, as they contain carefully designed tasks that test real capabilities, but that their *environments* must be redesigned to prevent this type of exploitation. Multifactorial variability (Principle M, Section 3) breaks the replay equivalence by ensuring the initial state differs across evaluations along multiple independent axes.

2.2 Point estimates and $\text{pass}@k$ as fragile CUA evals foundation

The environmental vulnerability above is compounded by methodological failures in how results are reported and aggregated. We identify three specific failures:

Failure 1: Treating tasks as i.i.d. trials. A benchmark with N tasks, each attempted once, is *not* equivalent to N coin flips. Tasks within the same application share UI flows, navigation patterns, and interaction primitives, and agent success is correlated across tasks that exercise similar capabilities (Agarwal et al., 2021). CIs that ignore this correlation dramatically understate uncertainty, leading to under-coverage rates exceeding 50% on realistic benchmark structures (Section 6.1).

Failure 2: Misuse of pass@ k . The pass@ k estimator (Chen et al., 2021) was designed for code generation, where an evaluator can sample k candidate programs, run each in isolation, and report whether *any* candidate passes the tests. This protocol is meaningful because unsuccessful candidates do not alter the problem instance, and the evaluator can select the successful artifact after the fact. CUA trajectories do not have this structure: a rollout is not a candidate artifact; it is an execution that changes application state. The operational quantity of interest is single-attempt success, not whether success appears at least once among k attempts. Moreover, as Remark 1 makes concrete: on a static benchmark, pass@ k is formally identical to the expected success rate of a replay policy that stores one successful trajectory and later replays it blindly. Reporting pass@ k in this setting measures the ability to find and reuse a successful trajectory, not reliable deployment capability.

Failure 3: Ignoring the nested data structure. CUA benchmarks have an inherently hierarchical structure: suites contain apps, apps contain scenarios, scenarios may be evaluated across configurations, and each configuration is attempted across rollouts. Collapsing this hierarchy into a single fraction discards information about *where* variability arises and produces confidence intervals that are either too narrow or too wide. We validate these concerns in Section 6.1 and Section 6.2.

3 Principles for Computer Use Environment Design

The diagnosis in Section 2 demonstrates that current benchmarks fail on two fronts: their environments are designed without principled safeguards, and their evaluation methodology does not withstand scrutiny. We begin the constructive part of the paper by proposing five design principles that a CUA benchmark environment must satisfy. Together, these principles form the **PRISM** framework: *privileged* verification, *realistic* environments, *integrity-checked* configurations, *sandboxed* execution, and *multifactorial* variability.

PRISM Design Principles

P — Privileged verification. Task completion must be verified as much as possible against the environment’s internal ground-truth state.

R — Realistic environments. Benchmark applications must faithfully reproduce the visual appearance, interaction patterns, and functional complexity of real-world applications.

I — Integrity-checked configurations. Every environment configuration must be automatically verified, as feasible (preconditions satisfied), coherent (task-specific data consistent with existing state), and non-trivial (task not already completed in the initial state).

S — Sandboxed execution. The environment must be fully self-contained, with no external dependencies, and must support deterministic reset to any specified initial state.

M — Multifactorial variability. The environment must vary independently along multiple axes relevant to the capabilities being measured, such as data content, visual appearance, and initial application state.

Each principle addresses a specific failure mode identified in Section 2. *Privileged verification* eliminates the biases and adversarial vulnerabilities of LLM-as-judge and screenshot-based verification (Zheng et al., 2023; Wang et al., 2026); privileged access to internal state makes verification deterministic and tamper-proof. *Realistic environments* ensure ecological validity, preventing the gap between toy interfaces and real-world deployment. *Integrity checking* is a structural necessity of any combinatorial benchmark: without it, a substantial fraction of generated configurations may be impossible or trivially pre-solved, silently corrupting aggregate metrics. *Sandboxed execution* ensures reproducibility: benchmarks that depend on live services produce results that drift with UI updates, A/B tests, data changes, and network or software interruptions. Finally, *multifactorial variability* prevents agents from succeeding by replaying memorized action sequences (Section 2.1) and enables fine-grained diagnostics by decomposing performance along each axis of variation.

Table 2 Evaluation of visual CUA benchmarks against the PRISM principles. ✓ = fully satisfies, ~ = partially satisfies, ✗ = does not satisfy. No existing benchmark satisfies all five principles; DIGIWorld is designed to be the first.

Benchmark	Platform	Privileged	Realistic	Integrity	Sandboxed	Multifactorial
AndroidWorld (Rawles et al., 2024)	Mobile	✓	✓	~	✓	~
OSWorld (Xie et al., 2024)	Desktop	✓	✓	~	✓	✗
VisualWebArena (Koh et al., 2024)	Web	✓	✓	✗	✓	~
WorkArena (Drouin et al., 2024)	Enterprise	✓	✓	✗	✗	~
REAL-bench (Garg et al., 2025)	Web	~	✓	✗	✓	✗
DigiData-Bench (Sun et al., 2025)	Mobile	✗	✓	✗	✗	✗
WebVoyager (He et al., 2024)	Web	✗	✓	✗	✗	✗
MobileWorld (Kong et al., 2025)	Mobile	✓	✓	~	✓	✗
OpenApps (Ullrich et al., 2025)	Mobile/Web	✓	~	✗	✓	✓
MiniWoB++ (Liu et al., 2018)	Web	✓	✗	✗	✓	~
DIGIWorld (ours)	Mobile	✓	✓	✓	✓	✓

3.1 Evaluating existing benchmarks against PRISM

Table 2 evaluates ten major visual CUA benchmarks against the PRISM criteria (Appendix L provides detailed per-benchmark analysis with rubric definitions). No existing benchmark satisfies all five principles. The most common gap is Integrity (I): while several benchmarks include initialization logic that provides partial safeguards, none really provides an automated offline pipeline that verifies every configuration is feasible, coherent, and non-trivial. This gap is especially dangerous in live or evolving environments where data drift can silently make tasks impossible. The Multifactorial (M) axis is also broadly absent: most benchmarks offer at most single-axis parameterization without independently varying visual appearance or data context.

4 From point estimates to principled aggregation

4.1 Sources of variability in CUA evaluation

CUA evaluation is characterized by two fundamentally different kinds of variability. **Action stochasticity** arises from the agent itself: given the exact same screen and task, different samples produce different action sequences. **Environmental variability** arises from the benchmark: different data, visual styles, or starting screens present the agent with different problems. Distinguishing these is critical because they answer different questions: “how reliably does the agent solve *this* problem?” versus “how robust is the agent *across* problems?”

Moreover, environments satisfying the PRISM principles feature stochasticity at multiple levels, which should be handled carefully by the statistical methodology used for agent evaluation. In this section, we develop an aggregation framework that properly handles performance estimation in such stochastic environments, and that fits the natural hierarchy of CUA benchmarks. The framework is general: it applies to any PRISM-compliant benchmark, not only to the one presented in this paper.

Any PRISM-compliant benchmark produces data with a nested hierarchy. At the coarsest level, a benchmark contains a suite of **apps** (A), each of which contains **scenarios** (S_a), i.e., task templates. Each scenario can be evaluated under multiple **configurations** ($C_{a,s}$), defined by the Cartesian product of whichever environmental axes the benchmark varies (e.g., data content, visual appearance, initial state). Finally, each configuration is attempted across multiple **rollouts** (R), which differ only in the agent’s stochastic sampling of actions. Each rollout produces a binary outcome $Y_{a,s,c,r} \in \{0, 1\}$.

A sensible choice for environmental axes for CUA are instance parameters, data profiles, themes, and UI states (see Appendix F). When all axes are disabled, there is exactly one configuration per scenario and the data reduce to a matrix of shape $A \times \bar{S} \times R$, analogous to the games $\times$ seeds structure in RL evaluation (Agarwal et al., 2021). As axes are enabled, the configuration space grows combinatorially and the data form a tree of depth four. An effective aggregation framework should handle both extremes and everything in between. We address this bottom-up: first the leaf level (per-configuration intervals, Section 4.2), then the full tree (suite-level aggregation, Section 4.3).

4.2 Wilson score intervals for binary outcomes

For a given configuration (a, s, c) with R rollouts, the empirical success rate is $\hat{p}_{a,s,c} = \frac{1}{R} \sum_{r=1}^R Y_{a,s,c,r}$. A common choice for computing a confidence interval for this success rate is the standard Wald interval $\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/R}$. However, the Wald interval has poor coverage at extreme success rates and small R (Brown et al., 2001): it collapses to a zero-width degenerate interval when $\hat{p} = 0$ or $\hat{p} = 1$. To avoid this, we propose to adopt the use of the Wilson score interval (Wilson, 1927), which is a simple algebraic transformation of the Wald interval that provides near-nominal coverage even for small R and extreme $\hat{p}$:

$$\hat{p}_W = \frac{\hat{p} + \frac{z^2}{2R}}{1 + \frac{z^2}{R}}, \quad W = \frac{z}{1 + \frac{z^2}{R}} \sqrt{\frac{\hat{p}(1-\hat{p})}{R} + \frac{z^2}{4R^2}} \quad (1)$$

where $z = z_{\alpha/2}$ is the standard normal quantile. The Wilson interval $[\hat{p}_W - W, \hat{p}_W + W]$ provides near-nominal coverage even for R as small as 3 and $\hat{p}$ near 0 or 1.

4.3 Hierarchical bootstrap

Suite-level aggregation. Collapsing all outcomes into a single fraction treats every rollout as an independent coin flip, ignoring the correlation structure diagnosed in Section 2.2. Following Agarwal et al. (2021), who showed that naïve averaging across domains (e.g. Atari games) produces unreliable RL agent rankings, we propose to aggregate in two stages. First, by computing a per-app success rate $\bar{p}_a$ by averaging over all scenarios, configurations, and rollouts within app a . Then, by summarizing the suite as $\hat{\theta} = \frac{1}{A} \sum_{a=1}^A \bar{p}_a$, the mean across apps. Because apps are a curated population rather than a random sample, we treat them as fixed and never resample them; this also ensures aggregation fairness, since each app contributes equally regardless of how many scenarios it contains.

Confidence intervals via hierarchical bootstrap. Standard bootstrap CIs that resample only at the rollout level dramatically understate uncertainty because they miss the variance contributed by scenario difficulty and configuration effects (Section 2.2, Failure 3). We therefore propose a hierarchical bootstrap that resamples at each active level *within* each app: (1) resample S_a scenarios with replacement, (2) within each scenario, resample along each enabled environmental axis independently, and (3) within each configuration, resample R rollouts. In the absence of varied environment configurations, steps (1) and (3) remain and step (2) becomes a no-op, reducing to a stratified bootstrap analogous to `rliable` with `task_bootstrap=True` (Agarwal et al., 2021), but applied independently within each app. For each replicate b , we recompute $\hat{\theta}^{(b)}$ and construct the 95% CI from the bootstrap quantiles. We provide a simulation study in Section 6.1 that confirms all resampling levels are necessary (e.g., omitting scenario resampling produces under-coverage).

5 DigiWorld: Following PRISM for Mobile Computer Use

We now present DIGIWORLD, a benchmark designed to satisfy all five PRISM principles: it is *realistic*, with each application faithfully reproducing real-world interaction patterns; it is *sandboxed*, being fully self-contained with no external dependencies and featuring deterministic reset via internal application state modification; it is *multifactorial*, supporting independent variation of data profiles, themes, UI states, and instance parameters; it is *privileged*, allowing verification via parameterized SQL queries against internal application state; it is *integrity-checked*, running automated feasibility checks and triviality filters to ensure every configuration is well-posed.

DIGIWORLD is made up of 15 hand-built, full-featured Android applications spanning 7 real-world domains. It comprises 387 scenarios (304 original single-app tasks plus 83 composed multi-step tasks) with over 4,300 task instances and a framework for defining and verifying tasks. Its hierarchical structure plugs directly into the evaluation methodology of Section 4.

Fifteen apps across seven domains. The 15 applications span 7 real-world domains (Table 4, Appendix E): communication (Email, Messaging), financial services (Payment, Banking), shopping (E-Commerce, QwikShop, Auction), entertainment (Music, Video), on-demand services (Eats, Ryde), travel (Flight Booking, Transit), and smart living (Smart Home, Parking). Each application is a fully-featured React Native Android application,

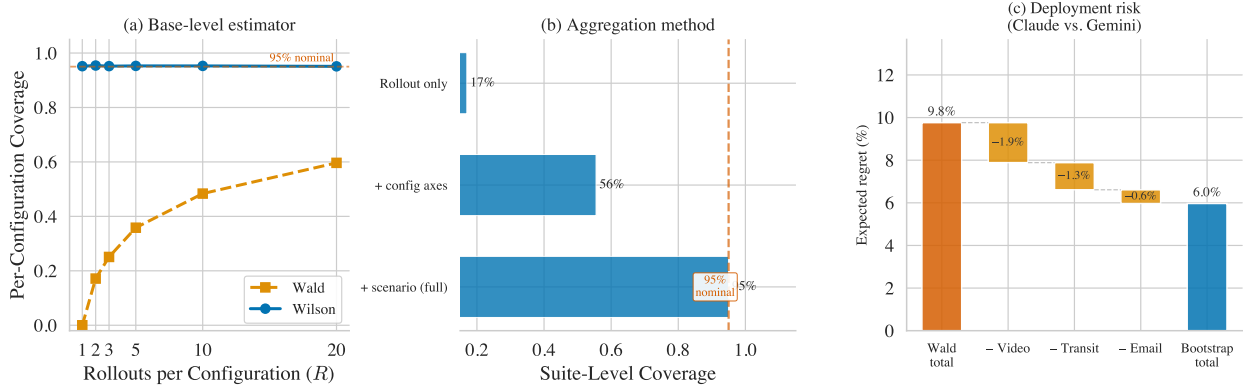


Figure 1 Validating the evaluation framework. **(a)** Per-configuration coverage of Wald vs. Wilson intervals across rollout budgets R , with parameters calibrated from DIGI WORLD evaluations (Appendix M). Wilson achieves near-nominal (95%) coverage at all R ; Wald under-covers severely, reaching only 25% at $R=3$. **(b)** Suite-level coverage under heterogeneous conditions. Each rung cumulatively adds one resampling level; only the full hierarchical bootstrap reaches 95%. **(c)** Per-app deployment regret (Claude vs. Gemini, Section 6.2). Expected regret is the probability of deploying the wrong model times the true per-app gap. The three apps where only Wald CIs claim significance contribute 3.8% of expected regret that the bootstrap prevents by correctly flagging them as uncertain.

hand-built for this benchmark, with a local database for the data state of the application, a JSON file representing its UI state, and a UI theming system.

The scenario lifecycle. A *scenario* defines a parameterized task template (e.g., “Send \$<amount> to <recipient>”) together with its verification logic. Each scenario is instantiated into multiple *instances* by binding concrete values to the parameters; each instance is compatible with a subset of data profiles, determined by automated feasibility checking (described below and detailed in Appendix K). A *configuration* is the combination of an instance with a specific data profile, theme, and UI state. The lifecycle has three phases: (1) *reset* (inject the selected profile’s database and instance-specific mockdata), (2) *execution* (the agent interacts through the standard Android GUI via screen-based actions (tap, type, scroll, with no programmatic API or tool access), and (3) *verification* (compare final application state to initial state). DIGI WORLD is agent-agnostic: any system that can send GUI events and receive screenshots can be evaluated. Appendix G provides full details.

Integrity at scale. Multifactorial variation generates a vast configuration space (over 3.2 million configurations across the suite), making manual curation of valid combinations infeasible. Programmatic integrity checking is therefore a *structural necessity* of any combinatorial benchmark. DIGI WORLD addresses this through three automated pipelines that run offline before any agent is evaluated: (1) *coherence checking*, which resolves template placeholders in instance-specific mockdata against each profile’s actual database to ensure referential consistency; (2) *feasibility checking*, which evaluates parameter-dependent constraints (e.g., sufficient balance for a transfer) to produce per-instance lists of compatible profiles; and (3) *triviality filtering*, which excludes configurations where the task is already completed in the initial state. Together, these mechanisms enable the combinatorial design to scale without sacrificing integrity guarantees. Appendix K provides implementation details.

6 Experiments

6.1 Validating the bootstrap via simulation

Our evaluation methodology makes two complementary improvements: a better base-level estimator (Wilson score intervals) and a hierarchical aggregation that respects the nested data structure.

Base-level estimator. Figure 1a compares per-configuration coverage of Wald and Wilson intervals. True success probabilities are calibrated from DIGI WORLD evaluation data, which is heavily bimodal (~68% of

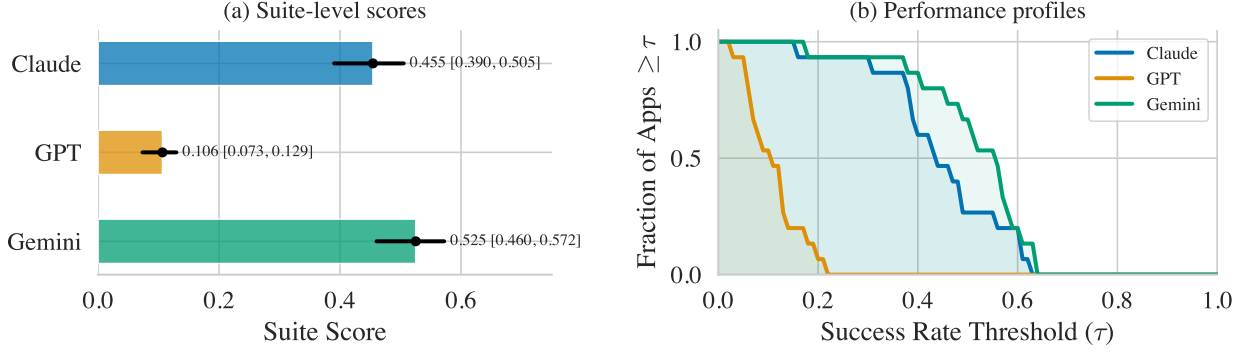


Figure 2 (a) Suite-level scores with 95% hierarchical bootstrap CIs. Claude and Gemini overlap; GPT is clearly separated. (b) Performance profiles: each curve shows the fraction of apps on which a model achieves at least the success rate τ .

configurations at 0% success, $\sim 32\%$ at 100%); we recover continuous true probabilities via a Jeffreys posterior (Appendix M). At $R=3$ (the typical budget in CUA evaluation), the Wald interval covers the true value only 25% of the time, because it produces degenerate zero-width CIs when $\hat{p} = 0$ or $\hat{p} = 1$. The Wilson interval maintains near-nominal coverage (95%) across all R , including $R=1$, by producing non-degenerate intervals at the extremes.

Hierarchical aggregation. Figure 1b presents a cumulative *ladder* of bootstrap variants under heterogeneous conditions, with app-level success rates and noise parameters calibrated from DIGIWorld evaluations (Appendix M). A bootstrap that resamples only rollouts achieves just 17% suite-level coverage. Adding configuration-axis resampling (capturing uncertainty about which environments the agent encounters) raises coverage to 56%. Adding scenario resampling on top completes the full hierarchy at 95% coverage, the only variant to reach the nominal level. Each level of the hierarchy captures genuine uncertainty that simpler methods miss.

6.2 Experimental results

Evaluation protocol. For each of the 387 scenarios, we evaluate $R = 20$ rollouts per sampled configuration. The full configuration space exceeds 3.2 million; we sample configurations by drawing instances, data profiles, themes, and UI states from the available pools for each scenario. We use $B = 1,000$ bootstrap replicates for all confidence intervals. We evaluate three frontier models: Claude Opus 4.6, GPT-5.4, and Gemini 3 Pro. All models were accessed via their public APIs in April 2026. Each model receives a task description, the current screenshot, and a set of available GUI actions (tap, type, scroll, press home/back, answer). No in-context demonstrations are provided.

Aggregate results. Figure 2a shows suite-level scores with 95% hierarchical bootstrap CIs. Claude and Gemini achieve similar scores with broadly overlapping CIs, indicating that the difference is not statistically meaningful; GPT struggles with mobile computer use as reported by previous work (Kong et al., 2025). Figure 2b shows performance profiles (Agarwal et al., 2021) across apps. Per-app results (Appendix I) show wide CIs even with thousands of rollouts, underscoring that app-level variance dominates the uncertainty budget. Results for additional model variants are in Appendix J.

Would naïve statistics get it wrong? Because Wald CIs underestimate uncertainty, they declare statistically significant differences for apps where the true gap is small and the ranking is fragile. To measure the downstream cost, we define *expected regret* per app as the probability of selecting the inferior model (estimated via 500 split-half simulations at one configuration per scenario) multiplied by the true performance gap for that app. A practitioner who deploys whichever model a CI method declares “significantly better” incurs this regret on every app where the method is both overconfident and wrong. Figure 1c shows that the hierarchical bootstrap reduces total expected regret from 9.8% to 6.0% by correctly withholding significance on three apps whose rankings are too uncertain to act on. The 3.8% of prevented regret is comparable to the gap between

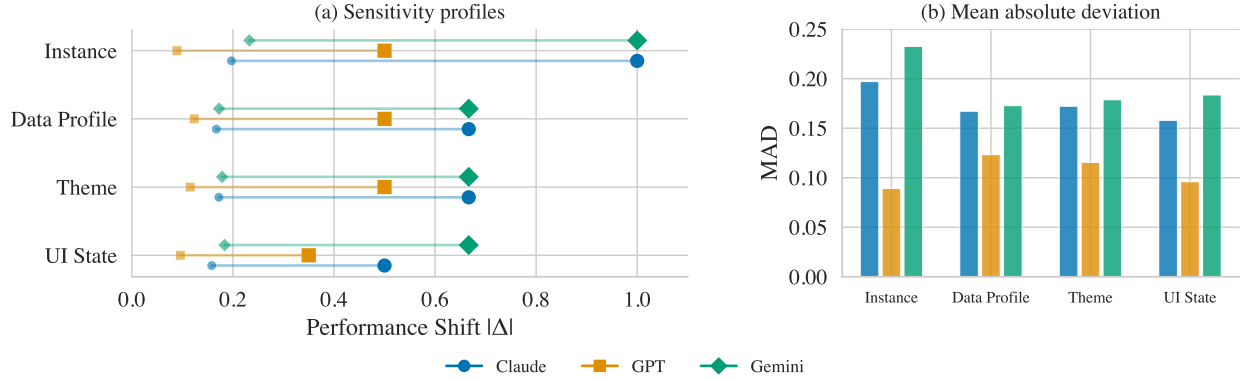


Figure 3 (a) Sensitivity profiles: small marker = MAD (average instability), large marker = 90th percentile of $|\Delta|$ (worst case); longer lines indicate heavier tails. (b) MAD per axis and model.

adjacent frontier models on current benchmarks, illustrating how miscalibrated CIs can silently erase real performance differences.

Variability decomposition. The combinatorial structure of DIGIWORLD enables diagnostic analyses not possible on static benchmarks. We use matched-pair comparisons to isolate the effect of each environmental axis, summarizing instability via the mean absolute deviation (MAD) of performance shifts along axes (Appendix H). Figure 3 shows that all axes produce MADs in the range 0.09–0.21, with no single axis dominating uniformly. This heterogeneity confirms that multifactorial variation is essential: a benchmark varying only one axis would miss substantial sources of instability.

Replay resistance. Returning to the replay experiment of Section 2.1, a replay agent only succeeds on DIGIWORLD (Table 1) in simple navigation tasks (e.g., “show contacts list,” “logout”) where the target UI element occupies a fixed position regardless of configuration. Instead, tasks requiring data-dependent reasoning (e.g., filtering, searching) fail almost completely. This confirms that the variability selectively defeats memorization while preserving solvability for genuine visual reasoning.

7 Related work

CUA benchmarks. The past two years have seen a proliferation of CUA benchmarks. OSWorld (Xie et al., 2024) provides tasks in a virtualized Ubuntu environment. AndroidWorld (Rawles et al., 2024) defines tasks on live Android applications. WebArena (Zhou et al., 2024) and VisualWebArena (Koh et al., 2024) focus on web-based tasks. WorkArena (Drouin et al., 2024) targets enterprise software. REAL-bench (Garg et al., 2025) constructs deterministic replicas of popular websites, WebVoyager (He et al., 2024) tests agents on live production websites, DigiData-Bench (Sun et al., 2025) evaluates mobile control agents on live apps, MobileWorld (Kong et al., 2025) benchmarks mobile agents with MCP-augmented tools, OpenApps (Ullrich et al., 2025) uses synthetic apps with explicit appearance variation, and MiniWoB++ (Liu et al., 2018) provides simplified web interaction tasks. While these benchmarks vary in scope and domain, they share common structural limitations; Table 2 evaluates each against the PRISM criteria defined in Section 3.

Overfitting, exploitation, and statistical methodology. The risk of agents exploiting fixed environments is well-documented in games and robotics (Zhang et al., 2018; Cobbe et al., 2020; Tobin et al., 2017), and environment unreliability has been observed for CUA (Wang et al., 2026; Stein et al., 2026). On the statistical side, Agarwal et al. (2021) showed that point estimates without robust aggregation produce unreliable agent rankings, proposing a framework with stratified bootstrap CIs. Our work extends this line of work to CUA, where the data hierarchy is deeper and binary outcomes require different estimators. Specifically, we use Wilson score intervals (Wilson, 1927; Brown et al., 2001; Capuano, 2026) and a hierarchical bootstrap to correctly handle the nested structure.

8 Limitations, broader impact, and conclusion

Limitations. PRISM operates at the level of environment and benchmark design; it does not address lower-level concerns such as harness design, task-specific verifier engineering, or the practical challenges of sandboxing complex environments. These are essential but can only be addressed reasonably atop principled foundations. DIGIWORLD currently targets only mobile (Android) applications; extending PRISM to desktop and web environments requires significant engineering effort, as each platform demands its own sandboxing strategy, verification mechanisms, and application framework.

Broader impact. We hope our work can redirect effort from benchmark-specific optimizations that may not transfer to deployment toward genuine understanding of agent capabilities. The variability decomposition framework provides a template for understanding *why* agents fail, not just *how often*.

We have shown that the CUA evaluation ecosystem suffers from two compounding pathologies: non-principled environment designs and evaluation methodologies. We address both through the PRISM design principles instantiated in DIGIWORLD, and through a hierarchical aggregation framework that produces confidence intervals by correctly modeling the nested data structure. The PRISM framework is not specific to DIGIWORLD: any interactive benchmark that satisfies these five principles can use the same hierarchical aggregation methodology. Together, these contributions provide the methodological foundation for CUA evaluation that is reliable, diagnostic, and resistant to gaming.

References

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, pages 29304–29320, 2021. https://proceedings.neurips.cc/paper_files/paper/2021/file/f514cec81cb148559cf475e7426eed5e-Paper.pdf.
- Lawrence D. Brown, T. Tony Cai, and Anirban DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2):101–133, 2001. doi: 10.1214/ss/1009213286.
- Francesco Capuano. A primer on measuring the uncertainty of success rates. Blog post, March 2026. <https://fracapuano.github.io/blog/success-rates>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.
- Karl Cobbe, Chris Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 2048–2056. PMLR, 13–18 Jul 2020. <https://proceedings.mlr.press/v119/cobbe20a.html>.
- Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks?, 2024. <https://arxiv.org/abs/2403.07718>.
- Divyansh Garg, Shaun VanWeelden, Diego Caples, Andis Draguns, Nikil Ravi, Pranav Putta, Naman Garg, Tomas Abraham, Michael Lara, Federico Lopez, James Liu, Atharva Gundawar, Prannay Hebbar, Youngchul Joo, Jindong Gu, Charles London, Christian Schroeder de Witt, and Sumeet Motwani. Real: Benchmarking autonomous agents on deterministic simulations of real websites. *arXiv preprint arXiv:2504.11543*, 2025.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models, 2024.

- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905, Bangkok, Thailand, aug 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.50. <https://aclanthology.org/2024.acl-long.50>.
- Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, Zhidan Liu, Steven Hoi, and Yue Wang. Mobileworld: Benchmarking autonomous mobile agents in agent-user interactive and mcp-augmented environments, 2025.
- Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration. In *International Conference on Learning Representations*, 2018. <https://arxiv.org/abs/1802.08802>.
- Christopher Rawles, Sarah Clinckemallie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents, 2024.
- Adam Stein, Davis Brown, Hamed Hassani, Mayur Naik, and Eric Wong. Detecting safety violations across many agent traces. *arXiv preprint arXiv:2604.11806*, 2026.
- Yuxuan Sun, Manchen Wang, Shengyi Qian, William R. Wong, Eric Gan, Pierluca D’Oro, Alejandro Castillejo Munoz, Sneha Silwal, Pedro Matias, Nitin Kamra, Satwik Kottur, Nick Raines, Xuanyi Zhao, Joy Chen, Joseph Greer, Andrea Madotto, Allen Bolourchi, James Valori, Kevin Carlberg, Karl Ridgeway, and Joseph Tighe. Digidata: Training and evaluating general-purpose mobile control agents, 2025.
- Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017. doi: 10.1109/IROS.2017.8202133.
- Karen Ullrich, Jingtong Su, Claudia Shi, Arjun Subramonian, Amir Bar, Ivan Evtimov, Nikolaos Tsilivis, Randall Balestrieri, Julia Kempe, and Mark Ibrahim. Openapps: Simulating environment variations to measure ui-agent reliability. *arXiv preprint arXiv:2511.20766*, 2025.
- Hao Wang, Qiuyang Mang, Alvin Cheung, Koushik Sen, and Dawn Song. How we broke top ai agent benchmarks: And what comes next. Blog post, April 2026. <https://rdi.berkeley.edu/blog/trustworthy-benchmarks-cont/>. Accessed on 2026-04-16.
- Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024.
- Chiyuan Zhang, Oriol Vinyals, Remi Munos, and Samy Bengio. A study on overfitting in deep reinforcement learning, 2018. <https://arxiv.org/abs/1804.06893>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024. <https://openreview.net/forum?id=oKn9c6ytLx>.

Appendix

A Proof of Replay Equivalence

Proof of Remark 1. Part 1: $\mathbb{E}[\text{SR}(\pi_R)] = \text{pass@}k(\pi)$. Fix a task i . The k rollouts are independent Bernoulli trials with success probability p_i . The replay policy π_R succeeds on task i if and only if at least one rollout succeeded (i.e., a successful trajectory exists to replay). Because the benchmark is deterministic and initializes task i to the same state s_i^0 every time, replaying a previously successful action sequence is guaranteed to succeed again. Therefore:

$$P(\pi_R \text{ succeeds on task } i) = 1 - (1 - p_i)^k.$$

Averaging over all N tasks gives $\mathbb{E}[\text{SR}(\pi_R)] = \text{pass@}k(\pi)$.

Part 2: $\text{SR}(\pi) \leq \text{pass@}k(\pi)$. It suffices to show $p_i \leq 1 - (1 - p_i)^k$ for each task. Let $q = 1 - p_i \in [0, 1]$. We need $q^k \leq q$. For $q \in [0, 1]$ and $k \geq 1$, this holds because $q^k = q \cdot q^{k-1} \leq q \cdot 1 = q$, with strict inequality when $q \in (0, 1)$ and $k \geq 2$. $\square$

Proof of Corollary 2. For any $p_i > 0$, $(1 - p_i)^k \rightarrow 0$ as $k \rightarrow \infty$. Therefore $\mathbb{E}[\text{SR}(\pi_R)] = \frac{1}{N} \sum_{i=1}^N [1 - (1 - p_i)^k] \rightarrow 1$. $\square$

B Replay agent evaluation details

For each benchmark in Table 1, the replay agent stores successful trajectories from the best-performing frontier model on that benchmark: Claude Opus 4.6 for OSWORLD and DIGIWORLD, and Gemini 3 Pro for MOBILEWORLD. The “frontier model” row in the table reports that same model’s success rate.

B.1 MobileWorld

From the 201 tasks in MOBILEWORLD, we evaluate on a subset of 96 tasks. We exclude 105 tasks that our evaluation infrastructure cannot reliably support or that are instrumented inconsistently in MOBILEWORLD itself. Specifically: (i) 40 tasks tagged **agent-mcp** require external MCP cloud APIs (Amap for maps/routing, Stockstar for financial data, arXiv/GitHub for information retrieval) for which we lack credentials; (ii) 41 Mastodon tasks and 16 Mattermost tasks rely on Docker-in-Docker backend services which were unstable on our virtual machines; and (iii) 8 mall shopping tasks fail deterministically across every run of every model with “No callback data found,” indicating a broken callback mechanism in the app. The remaining 96 tasks span 7 app categories (Calendar, Mail, Messages, Maps, Chrome, Files/Docreader, Taodian, Camera/Gallery, Clock, Settings, Contacts) and cover the full range of MOBILEWORLD’s task complexity, including web-browsing tasks that require live internet access.

C Cost considerations

The combinatorial evaluation protocol we propose is substantially more expensive than evaluating on a static benchmark: over 8,000 configurations per scenario vs. 1. We argue that this cost is the price of *meaningful* evaluation. A cheap evaluation that does not measure what it claims to measure is not a bargain. Repeating rollouts on a single fixed configuration yields a tight confidence interval on *that configuration’s* success rate, but this is not the quantity of interest: what matters is the agent’s expected performance across the full space of realistic conditions. The combinatorial design measures this more meaningful quantity by sampling diverse configurations, at the cost of more total rollouts. In practice, random sampling of configurations (rather than exhaustive enumeration) keeps the cost tractable: $387 \text{ scenarios} \times R=20 \text{ rollouts} = 7,740 \text{ rollouts per model}$, parallelizable across containers to a few hours of wall time.

D Additional simulation results

D.1 Full coverage tables

Table 3 reports the complete simulation results across all combinations of rollout counts and resampling strategies, including the heterogeneous within-app condition.

Table 3 Extended simulation results ($S=10$ scenarios/app, $3 \times 3 \times 3$ configurations, 95% CI). Under heterogeneous within-app noise ($\sigma_{\text{scen}} = 0.08$, $\sigma_{\text{config}} = 0.03$), rollout-only coverage collapses to 60%.

Condition	Resampling	Coverage	Width
Homogeneous, $R=3$	Scen + Config + Roll	1.000	0.061
	Scen + Config	1.000	0.046
	Scen + Roll	1.000	0.024
	Roll only	0.860	0.013
Heterogeneous, $R=3$	Scen + Config + Roll	1.000	0.065
	Roll only	0.600	0.013

D.2 Sensitivity to number of bootstrap replicates

We test $B \in \{100, 300, 500, 1000, 2000\}$ bootstrap replicates for the full hierarchical bootstrap with $R = 3$. Coverage remains at 100% for all values of $B \geq 100$. CI width stabilizes at $B = 300$ (change < 0.001 for larger B), justifying our default choice of $B = 300$ for simulations and $B = 1,000$ for final evaluations.

E Detailed DigiWorld statistics

Table 4 The DIGI WORLD application suite. 15 apps across 7 domains, each with ~ 10 data profiles, ~ 10 themes, and ~ 10 UI states.

Domain	Applications	Task types	Scenarios
Communication	Email, Messaging	Send, search, organize	39
Financial	Payment, Banking	Transfer, check balance, manage accounts	64
Shopping	E-Commerce, QwikShop, Auction	Browse, purchase, bid, review	115
Entertainment	Music, Video	Play, search, manage playlists	42
On-Demand	Eats, Ryde	Order food, book rides	34
Travel	Flight Booking, Transit	Search flights, plan routes	49
Smart Living	Smart Home, Parking	Control devices, find parking	44
Total (original + composed)			387

F Sources of variability in DigiWorld

G Scenario lifecycle details

A *scenario* in DIGI WORLD defines a parameterized task template (e.g., “Send \$<amount> to <recipient>”) together with its verification logic. Each scenario is instantiated into multiple *instances* by binding concrete values to the template parameters (e.g., “Send \$50 to Alice,” “Send \$200 to Bob”). Instance randomization (Table 6) controls whether a fixed default instance or a randomly selected instance is used at evaluation time. Each instance is compatible with a subset of the available data profiles, determined by automated feasibility checking (Section 5). A *configuration* is then the combination of an instance with a specific data profile, theme, and UI state drawn from whichever environmental axes are enabled. The scenario lifecycle proceeds in three phases:

Table 5 Per-app scenario counts and task type distribution in DIGIWORLD. Action tasks require the agent to modify application state; information retrieval (IR) tasks require the agent to extract and report information.

Application	Action	IR	Total
Email	12	7	19
Messaging	19	1	20
Payment	16	11	27
Banking	19	18	37
E-Commerce	18	40	58
QwikShop	22	2	24
Auction	27	6	33
Music	14	0	14
Video	12	16	28
Eats	9	5	14
Ryde	11	9	20
Flight Booking	7	10	17
Transit	13	19	32
Smart Home	21	6	27
Parking	17	0	17
Total	237	150	387

Table 6 The five sources of variability in DIGIWORLD. The four environmental axes are independently toggleable; rollout stochasticity is always present. When all environmental axes are disabled, the benchmark is deterministic and the only uncertainty comes from the agent’s sampling.

Source	Type	What varies	Example
Instance params	Environmental	Task parameters (amounts, recipients, items)	“Send \$50” vs. “Send \$200”
Data profile	Environmental	Database content (accounts, messages, history)	3 contacts vs. 50 contacts
Theme	Environmental	Visual styling (colors, fonts, layout density)	Light mode vs. dark mode
UI state	Environmental	Starting screen and navigation state	Home screen vs. settings page
Rollouts	Algorithmic	Agent’s stochastic sampling (seed, temperature)	Same screen, different actions

1. Reset. The framework selects a compatible profile, pushes the corresponding database and UI state files to the emulator via ADB, and injects instance-specific mockdata through a context extraction pipeline that resolves template placeholders against the profile’s database (Section 5).

2. Execution. The agent interacts with the application through the standard Android interface. DIGIWORLD is agent-agnostic: any system that can send touch/type events to an Android emulator and receive screenshots can be evaluated.

3. Verification. The framework captures the final application state and compares it to the initial state using parameterized SQL queries. For *action* tasks (e.g., sending an email), verification checks that the expected records were created or modified. For *information retrieval* tasks (e.g., “What is the account balance?”), the framework extracts the ground-truth answer from the database and compares it to the agent’s response using flexible matchers that handle numeric formats, date representations, durations, and Unicode normalization.

H Variability decomposition: methodology and per-app details

For a given axis (e.g., theme), we construct matched pairs of configurations that differ only along that axis. Let $c_1 = (i, d, t_1, u)$ and $c_2 = (i, d, t_2, u)$ be two configurations with the same instance i , data profile d , and UI state u but different themes t_1 and t_2 . The performance difference $\Delta = \hat{p}_{c_1} - \hat{p}_{c_2}$ isolates the effect of the theme change. The same methodology applies symmetrically to all four environmental axes. We summarize

the instability along each axis using the **Mean Absolute Deviation (MAD)**:

$$\text{MAD} = \frac{1}{|\mathcal{P}|} \sum_{(c_1, c_2) \in \mathcal{P}} |\Delta_{c_1, c_2}| \quad (2)$$

which measures the average magnitude of performance shifts when a single axis changes, regardless of direction.

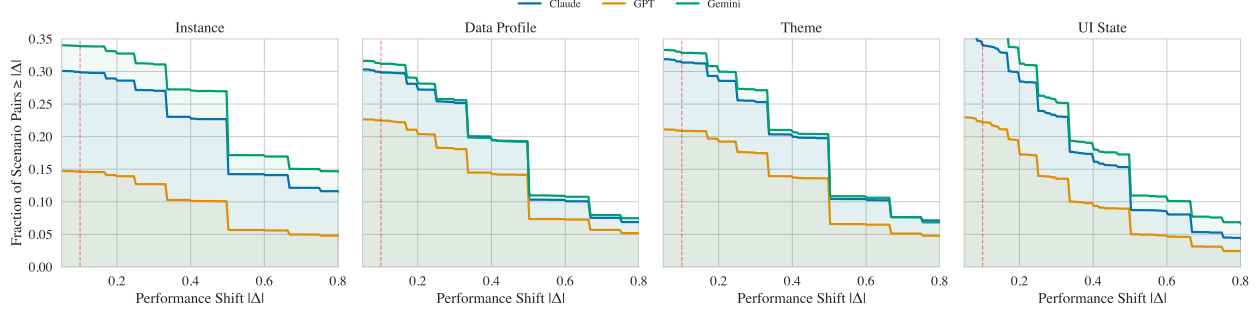


Figure 4 Exceedance curves for scenario-level performance shifts, per model and environmental axis. For each axis, we compute per-scenario success rates under each axis value (averaging across all other axes), take pairwise $|\Delta|$, and plot the fraction of pairs exceeding each threshold. Higher curves indicate greater sensitivity. The dashed red line marks $|\Delta|=0.10$.

Table 7 Per-app MAD (Mean Absolute Deviation) for Claude across each variability axis.

Application	Instance MAD	Data MAD	Theme MAD	UI State MAD
Email	.200	.190	.138	.170
Messaging	.165	.136	.133	.087
Payment	.146	.123	.145	.123
Banking	.156	.132	.116	.137
E-Commerce	.233	.222	.224	.211
QwikShop	.112	.073	.051	.061
Auction	.083	.133	.144	.129
Music	.249	.274	.203	.229
Video	.346	.300	.308	.244
Eats	.109	.104	.090	.065
Ryde	.140	.157	.118	.107
Flight	.162	.170	.156	.081
Transit	.290	.239	.249	.246
Smart Home	.247	.204	.223	.181
Parking	.154	.110	.114	.084
Mean	.186	.171	.161	.144

I Per-app performance

J Additional model results

We additionally evaluate two non-frontier model variants—Claude Sonnet 4.6 and Gemini 3 Flash—to provide a broader picture of performance across model tiers. Figure 6 shows suite-level scores and performance profiles for all five models.

K Integrity checking implementation details

This appendix provides the implementation details for the three integrity mechanisms described in Section 5.

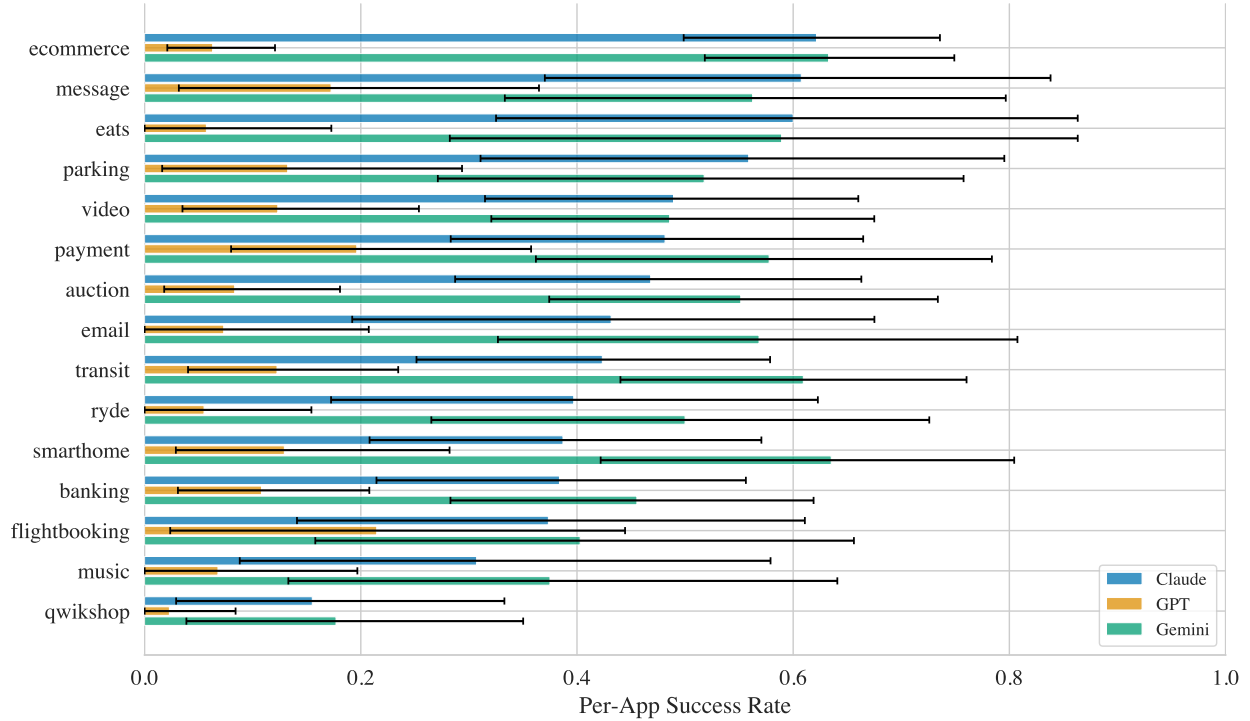


Figure 5 Per-app success rates for all three models, with 95% hierarchical bootstrap CIs. Apps are ordered by Claude’s performance.

K.1 Context extraction and grounded data injection

Mockdata files use `{{placeholder}}` tokens that are resolved against the profile’s SQLite database at reset time. Placeholders fall into three categories:

- **Identity templates** extract values matching the profile’s user identity: `{{current_user_email}}`, `{{current_user_id}}`, `{{current_user_name}}`.
- **Relational templates** extract values from existing records to ensure referential consistency: `{{first_room_id}}` resolves to an actual room ID in the smart home database, `{{context_sender:work}}` resolves to a contact classified as “work” in the email app.
- **Positioning templates** compute values *relative to* existing data: `{{middle_transaction_time}}` queries the transactions table, sorts by timestamp, and returns a time midway through the list. A positioning service implements strategies (**B**eginning, **M**iddle, **E**nd) that place injected records at natural positions within the existing data distribution.

Each application defines its own template resolver that extends a base resolver with app-specific templates (e.g., `{{safe_amount}}` in the payment app resolves to an amount guaranteed to be below the wallet balance). The resolver walks the mockdata structure recursively, resolving every placeholder before the data is merged into the profile’s database. The same mechanism resolves the *scenario context*, i.e., structured metadata (e.g., user PIN, account number) passed to the verification system for ground-truth comparison.

This design ensures that the *same* scenario instance produces *different* concrete tasks on different profiles: “find the most recent transaction” yields different amounts, dates, and descriptions depending on which profile’s transaction history is loaded.

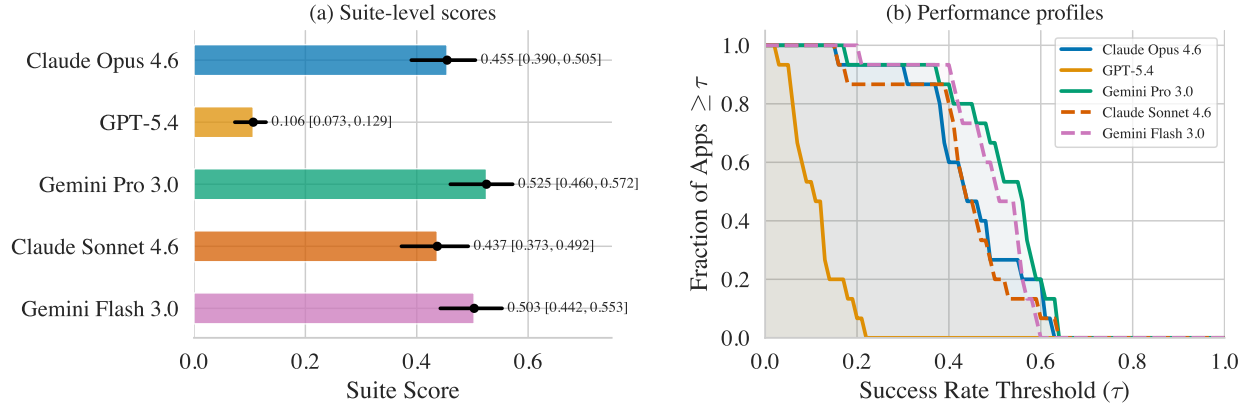


Figure 6 Suite-level scores and performance profiles for all five evaluated models. Solid lines in (b) denote the three frontier models from the main evaluation (Figure 2); dashed lines denote the additional non-frontier variants (Claude Sonnet 4.6 and Gemini 3 Flash).

K.2 Constraint-based feasibility checking

The feasibility pipeline evaluates each profile–instance pair offline and produces a per-instance list of compatible profiles. Four constraint types cover the space of preconditions:

- **EntityExists**: the profile’s database must contain at least n rows in a table, optionally filtered by user or column value (e.g., “at least 1 account for the current user”).
- **DataVolume**: a stricter variant requiring at least n rows matching a compound filter (e.g., “at least 2 completed transactions”).
- **Balance**: a numeric field must meet a threshold that can depend on the instance parameters (e.g., “wallet balance $\geq$ the transfer amount $<\text{amount}>$ ”).
- **MaxCount**: the profile must have *at most* n rows in a table, for scenarios that add records to capacity-limited collections (e.g., “fewer than 3 saved addresses”).

Beyond explicit constraints, the framework **auto-derives** additional constraints from the mockdata templates. If mockdata references $\{\{\text{first_room_id}\}\}$, the system automatically adds an **EntityExists** constraint requiring at least one room in the database. This closes the gap between what the templates *assume* exists and what the constraints *check* exists.

K.3 Triviality filtering

The framework executes the full verification logic against the *initial* state of each compatible profile, before the agent acts. Any profile where the task is already completed by coincidence (e.g., the “correct” database record happens to exist in the starting state) is excluded. This prevents inflated success rates from configurations where no agent action was needed.

L Detailed PRISM evaluation of existing benchmarks

This appendix provides a detailed assessment of each benchmark evaluated in Table 2. We focus exclusively on environments where the agent operates from visual inputs (screenshots), excluding text-only DOM-based benchmarks. To ensure consistent and transparent ratings, we apply the following rubric for each PRISM criterion:

Privileged (P): ✓ = all tasks verified via programmatic state inspection; ~ = mixed (some tasks use programmatic checks, others rely on LLM-as-judge); ✗ = no programmatic verification.

Realistic (R): ✓ = uses real-world or high-fidelity clone applications with production-grade complexity; ~ = purpose-built applications that capture realistic interaction patterns and visual complexity but are not clones of real products; ✗ = toy or heavily simplified interfaces that lack real-world visual or functional complexity.

Integrity (I): ✓ = automated offline pipeline verifies every configuration is feasible, coherent, and non-trivial; ~ = includes procedural initialization logic that sets up preconditions (e.g., clearing databases, injecting entities) but does not systematically verify feasibility, coherence, or non-triviality across all configurations; ✗ = no automated safeguards for configuration validity.

Sandboxed (S): ✓ = fully self-contained with deterministic reset; ~ = mostly isolated but with partial external dependencies; ✗ = relies on live services or lacks deterministic reset.

Multifactorial (M): ✓ = systematically varies ≥ 2 independent environmental axes (e.g., data content and visual appearance); ~ = single-axis parameterization (e.g., task parameters vary but UI and data context remain static); ✗ = completely static task set with no parameterization.

A critical, often overlooked dimension of Integrity is *staleness*: if an environment is live or its data is updated, tasks that reference specific entities, prices, or UI layouts may become impossible to complete. An integrity-checked benchmark must automatically verify that its tasks remain feasible and coherent against the current environment state.

L.1 AndroidWorld

Developed by Google DeepMind, AndroidWorld is a comprehensive environment for evaluating agents on Android devices (Rawles et al., 2024).

- **Privileged (✓):** Task success is verified programmatically by inspecting the device’s internal system state via the Android Debug Bridge (ADB) and UI element validation.
- **Realistic (✓):** Uses 20 real-world, unmodified Android applications, including OS-level apps and open-source apps from F-Droid.
- **Integrity-checked (~):** Tasks include dedicated initialization logic to set up preconditions (e.g., clearing databases, injecting required entities), providing procedural safeguards that reduce the chance of infeasible configurations. However, there is no automated offline pipeline that systematically verifies feasibility, coherence, and non-triviality across all randomly generated configurations before evaluation.
- **Sandboxed (✓):** Runs on a self-contained Pixel 6 emulator with a fixed Android 13 image, allowing deterministic resets of device state and time.
- **Multifactorial (~):** Dynamically constructs parameterized tasks by randomly generating entity values (names, phone numbers, dates) within natural language instructions. However, this is single-axis parameterization (task instance); it does not systematically vary independent axes like UI themes or initial app data states.

L.2 OSWorld

OSWorld tests multimodal agents on open-ended computer tasks across standard operating systems (Xie et al., 2024).

- **Privileged (✓):** Verification relies on custom execution-based evaluation scripts that programmatically check the internal system state (files, app state).
- **Realistic (✓):** Employs real, unmodified applications on standard operating systems (Ubuntu, Windows, macOS).
- **Integrity-checked (~):** Each task includes an initial state setup configuration (e.g., pre-configured VM snapshots, file system state) that establishes preconditions, reducing the chance of infeasible starting states. However, there is no automated offline pipeline that systematically verifies feasibility, coherence, or non-triviality of the 369 task configurations.

- **Sandboxed (✓):** Utilizes virtualization and containerization (e.g., Docker) for deterministic resetting and reproducibility without relying on live internet access.
- **Multifactorial (X):** Features a completely static test set of 369 fixed tasks. Each task has a fixed initial VM snapshot, a fixed goal, and a fixed evaluation script, with no parameterization or systematic variation.

L.3 VisualWebArena

An extension of the text-based WebArena, VisualWebArena introduces visually grounded tasks (Koh et al., 2024).

- **Privileged (✓):** Executes JavaScript within the browser to inspect the DOM and internal application state for programmatic verification.
- **Realistic (✓):** The underlying web applications (Classifieds, Shopping, Reddit) are functional clones of real platforms that faithfully reproduce the structural, visual, and interaction complexity of their production counterparts, populated with real or realistic data.
- **Integrity-checked (X):** Tasks are generated from curated templates without automated offline checks for feasibility, non-triviality, or staleness during generation.
- **Sandboxed (✓):** Fully sandboxed using Docker containers for each web application, enabling deterministic resets.
- **Multifactorial (~):** Uses parameterized task templates to generate unique tasks, but variability is restricted to swapping parameters within the same static UI and data environment.

L.4 WorkArena

WorkArena evaluates agents on enterprise software platforms (Drouin et al., 2024).

- **Privileged (✓):** Completion is verified programmatically using validation functions that check the internal state of the ServiceNow application.
- **Realistic (✓):** Uses the real, unmodified ServiceNow platform.
- **Integrity-checked (X):** Lacks a comprehensive, automated integrity check for all task configurations, relying instead on manual curation and LLM generation without strict offline validation against the live platform state.
- **Sandboxed (X):** The environment is remote-hosted and requires access to live ServiceNow instances, lacking deterministic reset or snapshotting capabilities.
- **Multifactorial (~):** Features thousands of parameterized task instances, but variability is limited to single-axis parameterization (e.g., different questions or forms) rather than independent axes of UI or state variation.

L.5 REAL-bench

REAL-bench tests agents on deterministic simulations of real-world websites (Garg et al., 2025).

- **Privileged (~):** Uses a hybrid approach: action-based tasks are verified via programmatic state-diff mechanisms, but information-retrieval tasks rely entirely on an LLM-as-judge.
- **Realistic (✓):** Uses high-fidelity deterministic replicas of popular websites (e.g., Airbnb, Amazon, Gmail) that faithfully mirror the structural and functional complexity of the production sites.
- **Integrity-checked (X):** Tasks are manually curated without automated verification for feasibility or coherence.
- **Sandboxed (✓):** A fully self-contained, controlled sandbox using deterministic simulations.
- **Multifactorial (X):** Focuses on standardized simulations without systematic variation across factors like UI themes or initial data states; tasks are fixed single-instance scenarios.

L.6 DigiData-Bench

DigiData-Bench evaluates general-purpose mobile control agents on complex real-world tasks (Sun et al., 2025).

- **Privileged (X):** Task success is evaluated either by human operators or via an LLM-as-judge analyzing screenshots and UI trees. There is no programmatic verification of the internal system state.
- **Realistic (✓):** Uses real-world Android applications on actual Android devices.
- **Integrity-checked (X):** Uses manual “state initialization instructions” for human operators to set up prerequisites, lacking an automated offline verification pipeline.
- **Sandboxed (X):** Runs on live devices interacting with applications in the wild, requiring human operators to monitor and intervene to prevent unsafe actions.
- **Multifactorial (X):** Consists of a fixed set of 309 goals without parameterized variation or independent axes of complexity.

L.7 WebVoyager

WebVoyager evaluates end-to-end web agents on real-world websites (He et al., 2024).

- **Privileged (X):** Relies entirely on GPT-4V as an LLM-judge to evaluate task success based on screenshots, with no programmatic state verification.
- **Realistic (✓):** Agents interact directly with live, real-world production websites (e.g., Apple, Booking.com, Google Flights).
- **Integrity-checked (X):** Tasks are manually compiled without automated feasibility or coherence checks against the live, constantly drifting websites.
- **Sandboxed (X):** Operates on the live internet, making the environment uncontrolled and susceptible to UI updates, A/B tests, and network latency.
- **Multifactorial (X):** Features a static set of 643 tasks without systematic variation.

L.8 MobileWorld

MobileWorld benchmarks autonomous mobile agents in agent-user interactive and MCP-augmented environments (Kong et al., 2025).

- **Privileged (✓):** Employs multiple deterministic verification methods, including textual answer verification, backend database queries (e.g., PostgreSQL for Mattermost), local storage inspection via ADB, and application callbacks, entirely avoiding LLM-as-judge for task success.
- **Realistic (✓):** Uses real Android applications, including a mix of stock apps and production-grade self-hosted open-source alternatives (Mattermost, Mastodon, Mall4Uni) that faithfully reproduce real-world functional complexity.
- **Integrity-checked (~):** Uses snapshot-based state management to ensure reproducible starting conditions, providing deterministic preconditions for each task. However, there is no automated offline pipeline to verify the feasibility, coherence, or non-triviality of the 201 manually created tasks.
- **Sandboxed (✓):** Fully containerized using Docker-in-Docker with rooted Android Virtual Devices and self-hosted backends, ensuring complete isolation from external dependencies and deterministic reset capabilities.
- **Multifactorial (X):** Features a static set of 201 fixed tasks, each starting from a predetermined snapshot. It does not parameterize tasks or systematically vary independent axes like UI theme or initial data.

L.9 OpenApps

OpenApps focuses on measuring UI-agent reliability by simulating environment variations (Ullrich et al., 2025).

- **Privileged (✓):** Reward and verification are based on programmatic checks of the full underlying application state.
- **Realistic (~):** Uses purpose-built synthetic applications (calendar, messenger, maps) that capture realistic interaction patterns and visual complexity, but are not clones of real-world products and lack the full functional depth of production applications.
- **Integrity-checked (X):** Tasks are generated based on templates without a separate, automated validation step for integrity checking.
- **Sandboxed (✓):** A self-contained Python-based environment running on a single CPU, allowing for scalable, deterministic resets.
- **Multifactorial (✓):** Explicitly designed to systematically vary appearance (e.g., dark theme, font size) and content independently to test agent generalization across thousands of app versions.

L.10 MiniWoB++

MiniWoB++ is a collection of simple web interaction environments (Liu et al., 2018).

- **Privileged (✓):** Controlled via Selenium WebDriver, providing full access to the DOM and page state for programmatic verification.
- **Realistic (X):** Uses synthetic toy interfaces and heavily simplified clones of web applications with surrogate backends.
- **Integrity-checked (X):** Tasks are manually created or templated without systematic integrity checks for each instance.
- **Sandboxed (✓):** Runs locally via Selenium without requiring internet access, ensuring deterministic resets.
- **Multifactorial (~):** Incorporates parameter randomization for task goals, but lacks systematic variation of visual appearance or data context as independent axes.

M Simulation calibration from evaluation data

Both simulation studies in Figure 1 are calibrated from DIGIWORLD evaluation data to ensure realistic conditions.

M.1 Base-level estimator calibration (Figure 1a)

We extract per-configuration success rates from all three frontier model evaluations on DIGIWORLD (~19,500 configurations total). The observed distribution is heavily bimodal: ~68% of configurations have a 0% success rate and ~32% have a 100% success rate, reflecting that most configurations are either very hard or very easy for current models at $R=3$ rollouts.

The observed rates are discretized (only 0, $\frac{1}{3}$, $\frac{2}{3}$, 1 are possible at $R=3$), so we recover continuous true probabilities via a Jeffreys posterior: for each observed k successes in R rollouts, we draw $p_{\text{true}} \sim \text{Beta}(k + \frac{1}{2}, R - k + \frac{1}{2})$. This standard noninformative posterior converts the degenerate point masses at 0 and 1 into realistic continuous distributions (e.g., a 0/3 observation yields $p_{\text{true}} \sim \text{Beta}(0.5, 3.5)$ with mean 0.125). We then simulate R fresh rollouts from each p_{true} and compute Wald and Wilson CIs.

M.2 Hierarchical bootstrap calibration (Figure 1b)

The simulation uses $A=15$ apps, $S=8$ scenarios per app, $3 \times 3 \times 3$ configurations per scenario, and $R=3$ rollouts. Three parameters are calibrated from evaluation data:

- **App-level success probabilities.** We use the observed per-app success rates from a representative mid-range model (ranging from 0.16 to 0.62, mean 0.41), rather than the synthetic $\text{linspace}(0.05, 0.95)$ used in prior work.
- **Scenario noise (σ_{scen}).** We compute the standard deviation of per-scenario success rates within each app, averaged across apps and models. The raw observed value (~ 0.39) is only slightly inflated by rollout sampling noise (the per-scenario rate averages over ~ 57 binary outcomes), yielding a corrected $\sigma_{\text{scen}} \approx 0.25$. This large value reflects the genuine difficulty spread within apps: some scenarios (e.g., “send an email”) are much easier than others (e.g., “find and forward a specific thread”).
- **Configuration noise (σ_{config}).** The observed per-configuration standard deviation within scenarios (~ 0.19) is almost entirely explained by rollout sampling noise at $R=3$ (expected $\sqrt{p(1-p)/R} \approx 0.28$), leaving a corrected $\sigma_{\text{config}} \approx 0.05$. This small value indicates that, at $R=3$, the dominant source of between-configuration variation is rollout stochasticity rather than true environmental effects, a finding consistent with the relatively modest MADs observed in Section 6.2. Larger R would resolve finer true environmental effects.

We run 200 independent experiments with $B=500$ bootstrap replicates for each of five methods (Wald CI, and four cumulative bootstrap variants).

Note on robust aggregation. For benchmarks with a larger number of applications or where some applications may produce anomalous scores (e.g., due to degenerate tasks or extreme difficulty mismatches), replacing the plain mean with a trimmed mean TM_α can improve robustness. In DIGIWORLD’s current setting ($A=15$ curated apps on a common 0–1 scale), the plain mean and 10% trimmed mean produce nearly identical results, so we use the simpler estimator. The hierarchical bootstrap framework supports any suite-level statistic, including trimmed means, medians, or interquartile means, as a drop-in replacement.

N Application gallery

DIGIWORLD comprises 15 purpose-built Android applications spanning five domains: communication (Email, Messaging), finance (Payment, Banking), shopping (E-Commerce, QwikShop, Auction), entertainment (Music, Video), on-demand services (Eats, Ryde), travel (Flight Booking, Transit), and smart living (Smart Home, Parking). Each application is a fully functional React Native app with local SQLite storage, custom theming support, and deep-link-driven state management. Figure 7 shows the default home screen of each application.

O Visual impact of variability axes

Figure 8 illustrates how each of the four environmental variability axes (Table 6) affects the visual appearance of four representative applications. For each app, we show the default configuration alongside variants produced by changing a single axis:

- **Instance:** different task parameters are bound to the scenario template, injecting different concrete data into the app (e.g., different email subjects and senders, different wallet balances, different restaurant and category names).
- **Data Profile:** loads an entirely different database profile, changing all visible content (e.g., different users, transaction histories, menu items).
- **Theme:** switches the visual styling to a dark palette, changing background colors, text contrast, and accent tones.
- **UI State:** starts the app on a different screen (e.g., Contacts, Transactions, Orders, Cards), altering the navigation context.

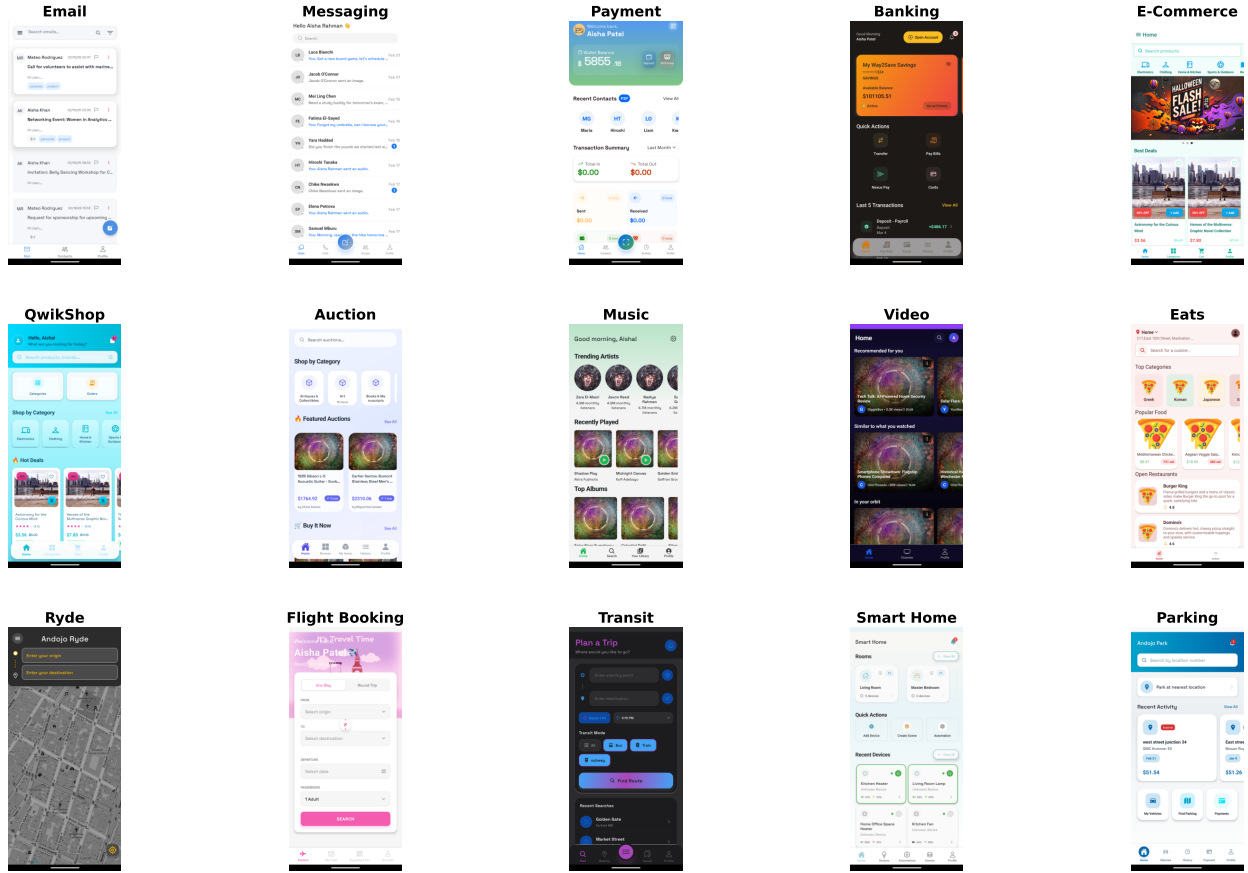


Figure 7 The 15 DigiWORLD applications shown in their default configuration. Each app is a self-contained, sandboxed Android application with realistic UI and data.

These visual differences are representative of the environmental variation that agents must handle robustly. All four axes are independently composable: a single evaluation configuration combines one instance, one data profile, one theme, and one UI state, yielding over 3.2 million verified unique configurations across the benchmark.

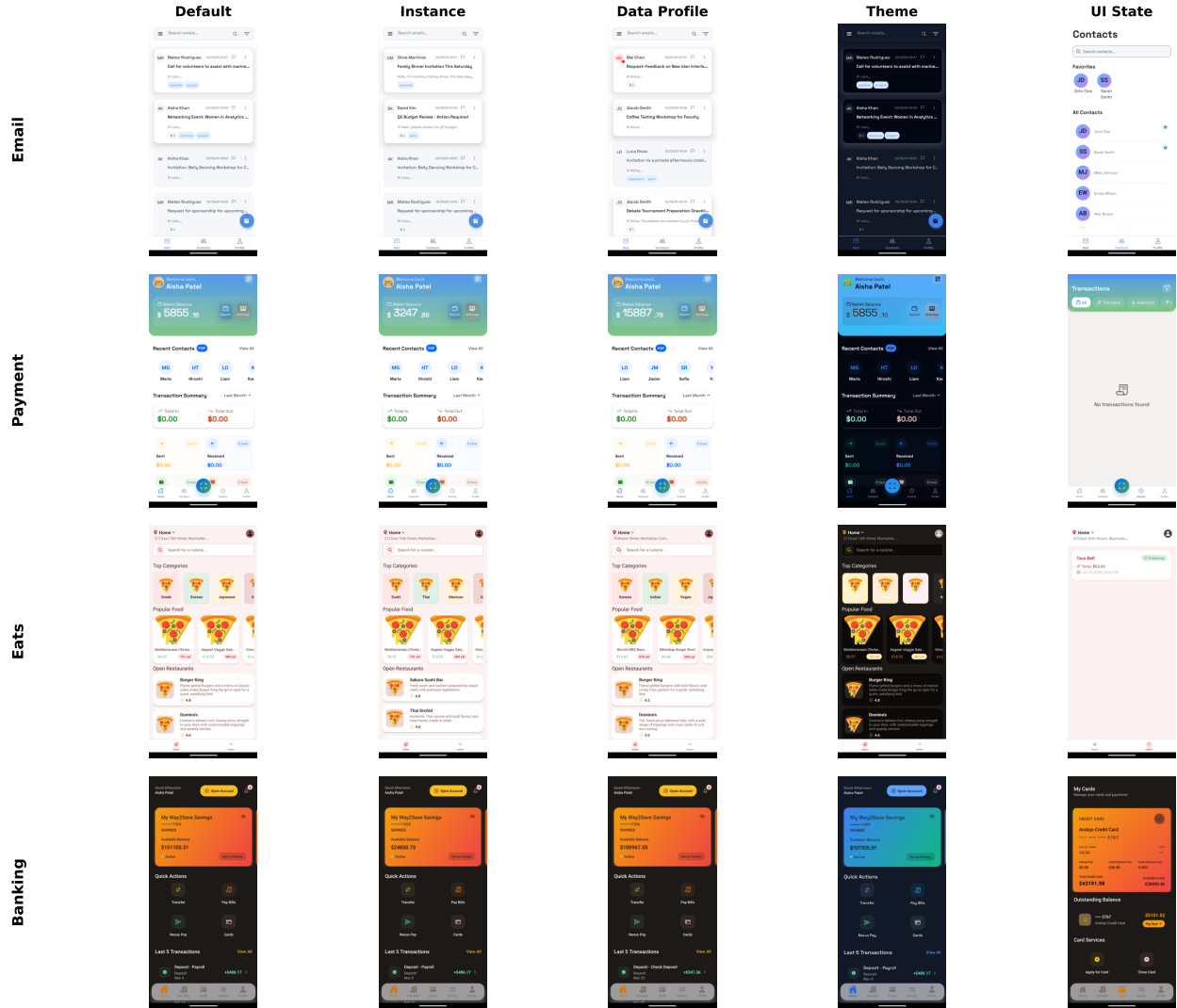


Figure 8 Visual impact of each environmental variability axis on four representative applications. Each column corresponds to one of the four environmental axes in Table 6; the leftmost column shows the default configuration for reference. Rows correspond to Email, Payment, Eats, and Banking.